

```
In [30]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random
```

NumPy Basics

```
In [31]: # 1. Create a NumPy array of numbers from 1 to 10 and print it.
a = np.array([1,2,3,4,5,6,7,8,9,10])
# a = np.arange(1,11)
a
```

```
Out[31]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
In [32]: # 2. Create a 3x3 matrix filled with zeros and another filled with ones.
b = np.zeros((3,3))
print(b)
b = np.ones((3,3))
print(b)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
In [33]: # 3. Generate an array of 5 random numbers between 0 and 1.

d = np.random.rand(5)
d
```

```
Out[33]: array([0.24379961, 0.84515025, 0.13845716, 0.60927878, 0.11326317])
```

```
In [34]: # 4. Find the sum, mean, minimum, and maximum of a NumPy array.

a = np.array([1,2,3,4,5])
sum = np.sum(a)
mean = np.mean(a)
minimum = np.min(a)
max = np.max(a)

print(f"Sum is {sum}")
print(f"Mean is {mean}")
print(f"Minimum is {sum}")
print(f"Sum is {sum}")
```

```
Sum is 15
Mean is 3.0
Minimum is 15
Sum is 15
```

```
In [35]: # 5. Reshape a 1D array of 12 elements into a 3x4 matrix.
```

```
a = np.array([1,2,3,4,5,6,7,8,9,10,11,12]).reshape(3,4)
a
```

```
Out[35]: array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])
```

```
In [36]: # 6. Multiply two arrays element-wise and print the result.
```

```
a = np.array([1,2,3,4,5])
b = np.array([6,7,8,9,10])
c = a * b
c
```

```
Out[36]: array([ 6, 14, 24, 36, 50])
```

```
In [37]: # 7. Slice the first 3 elements and the last 3 elements from an array.
```

```
a = np.array([1,2,3,4,5,6,7,8,9,10])
print(a[:3])
print(a[7:])
```

```
[1 2 3]
[ 8  9 10]
```

```
In [38]: # 8. Create an identity matrix of size 4x4 using NumPy.
```

```
print(np.identity(4))
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Pandas Basics

```
In [39]: # 1. Create a Pandas DataFrame with columns: Name, Age, City, and Salary.
```

```
data = {
    "Name": ['Sulav Pandey', 'Andrej Karpathy', 'Geoffrey Hinton',
             'Yann LeCun', 'Fei-Fei Li', 'Demis Hassabis'],
    "Age": [19, 39, 78, 65, 49, 49],
    "City": ['Kathmandu', 'San Francisco', 'Toronto',
             'New York', 'Stanford', 'London'],
    "Salary": [20000, 2500000, 1000000, 1500000, 800000, 3000000]
}

df = pd.DataFrame(data)
df
```

Out[39]:

	Name	Age	City	Salary
0	Sulav Pandey	19	Kathmandu	20000
1	Andrej Karpathy	39	San Francisco	2500000
2	Geoffrey Hinton	78	Toronto	1000000
3	Yann LeCun	65	New York	1500000
4	Fei-Fei Li	49	Stanford	800000
5	Demis Hassabis	49	London	3000000

In [40]: *# 2. Display the first 5 rows and last 3 rows of a DataFrame.*

```
df.head(5)
df.tail(3)
```

Out[40]:

	Name	Age	City	Salary
3	Yann LeCun	65	New York	1500000
4	Fei-Fei Li	49	Stanford	800000
5	Demis Hassabis	49	London	3000000

In [41]: *# 3. Check the shape, data types, and basic statistics of a DataFrame using info()*

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Name    6 non-null        object
1    Age      6 non-null        int64
2    City     6 non-null        object
3    Salary   6 non-null        int64
dtypes: int64(2), object(2)
memory usage: 320.0+ bytes
```

Out[41]:

	Age	Salary
count	6.000000	6.000000e+00
mean	49.833333	1.470000e+06
std	20.458902	1.111306e+06
min	19.000000	2.000000e+04
25%	41.500000	8.500000e+05
50%	49.000000	1.250000e+06
75%	61.000000	2.250000e+06
max	78.000000	3.000000e+06

In [42]: *# 4. Select a single column and multiple columns from a DataFrame.*

```
a = df[['Name']]
b = df[['Name', 'Age']]
print(a)
print(b)
```

```
      Name
0  Sulav Pandey
1  Andrej Karpathy
2  Geoffrey Hinton
3    Yann LeCun
4    Fei-Fei Li
5  Demis Hassabis
      Name  Age
0  Sulav Pandey  19
1  Andrej Karpathy  39
2  Geoffrey Hinton  78
3    Yann LeCun  65
4    Fei-Fei Li  49
5  Demis Hassabis  49
```

In [43]: *# 5. Filter rows where Salary is greater than 40000.*

```
high_salary = df[df['Salary']>40000]
print(high_salary)
```

```
      Name  Age  City  Salary
1  Andrej Karpathy  39  San Francisco  2500000
2  Geoffrey Hinton  78    Toronto  1000000
3    Yann LeCun  65    New York  1500000
4    Fei-Fei Li  49    Stanford   800000
5  Demis Hassabis  49    London  3000000
```

In [44]: *# 6. Add a new column 'Tax' which is 10% of Salary.*

```
Tax = df['Salary']*0.1
df['Tax'] = Tax
df
```

Out[44]:

	Name	Age	City	Salary	Tax
0	Sulav Pandey	19	Kathmandu	20000	2000.0
1	Andrej Karpathy	39	San Francisco	2500000	250000.0
2	Geoffrey Hinton	78	Toronto	1000000	100000.0
3	Yann LeCun	65	New York	1500000	150000.0
4	Fei-Fei Li	49	Stanford	800000	80000.0
5	Demis Hassabis	49	London	3000000	300000.0

In [45]: *# 7. Sort the DataFrame by Age in ascending order.*

```
df_sorted = df.sort_values(by='Age')
df_sorted
```

Out[45]:

	Name	Age	City	Salary	Tax
0	Sulav Pandey	19	Kathmandu	20000	2000.0
1	Andrej Karpathy	39	San Francisco	2500000	250000.0
4	Fei-Fei Li	49	Stanford	800000	80000.0
5	Demis Hassabis	49	London	3000000	300000.0
3	Yann LeCun	65	New York	1500000	150000.0
2	Geoffrey Hinton	78	Toronto	1000000	100000.0

In [46]: *# 8. Find and count missing values in each column using isnull().sum()*

```
df.isnull().sum()
```

```
Out[46]: Name      0
Age        0
City       0
Salary     0
Tax        0
dtype: int64
```

In [47]: *# 9. Replace missing values in a numeric column with the column mean.*

```
df.loc[2, 'Salary'] = np.nan
df.loc[4, 'Salary'] = np.nan
print(df)

df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df
```

	Name	Age	City	Salary	Tax
0	Sulav Pandey	19	Kathmandu	20000.0	2000.0
1	Andrej Karpathy	39	San Francisco	2500000.0	250000.0
2	Geoffrey Hinton	78	Toronto	NaN	100000.0
3	Yann LeCun	65	New York	1500000.0	150000.0
4	Fei-Fei Li	49	Stanford	NaN	80000.0
5	Demis Hassabis	49	London	3000000.0	300000.0

Out[47]:

	Name	Age	City	Salary	Tax
0	Sulav Pandey	19	Kathmandu	20000.0	2000.0
1	Andrej Karpathy	39	San Francisco	2500000.0	250000.0
2	Geoffrey Hinton	78	Toronto	1755000.0	100000.0
3	Yann LeCun	65	New York	1500000.0	150000.0
4	Fei-Fei Li	49	Stanford	1755000.0	80000.0
5	Demis Hassabis	49	London	3000000.0	300000.0

In [48]: *# 10. Group data by City and find the average Salary per City.*

```
avg_salary_by_city = df.groupby('City')['Salary'].mean()
print(avg_salary_by_city)
```

```
City
Kathmandu      20000.0
London        3000000.0
New York       1500000.0
San Francisco  2500000.0
Stanford       1755000.0
Toronto        1755000.0
Name: Salary, dtype: float64
```

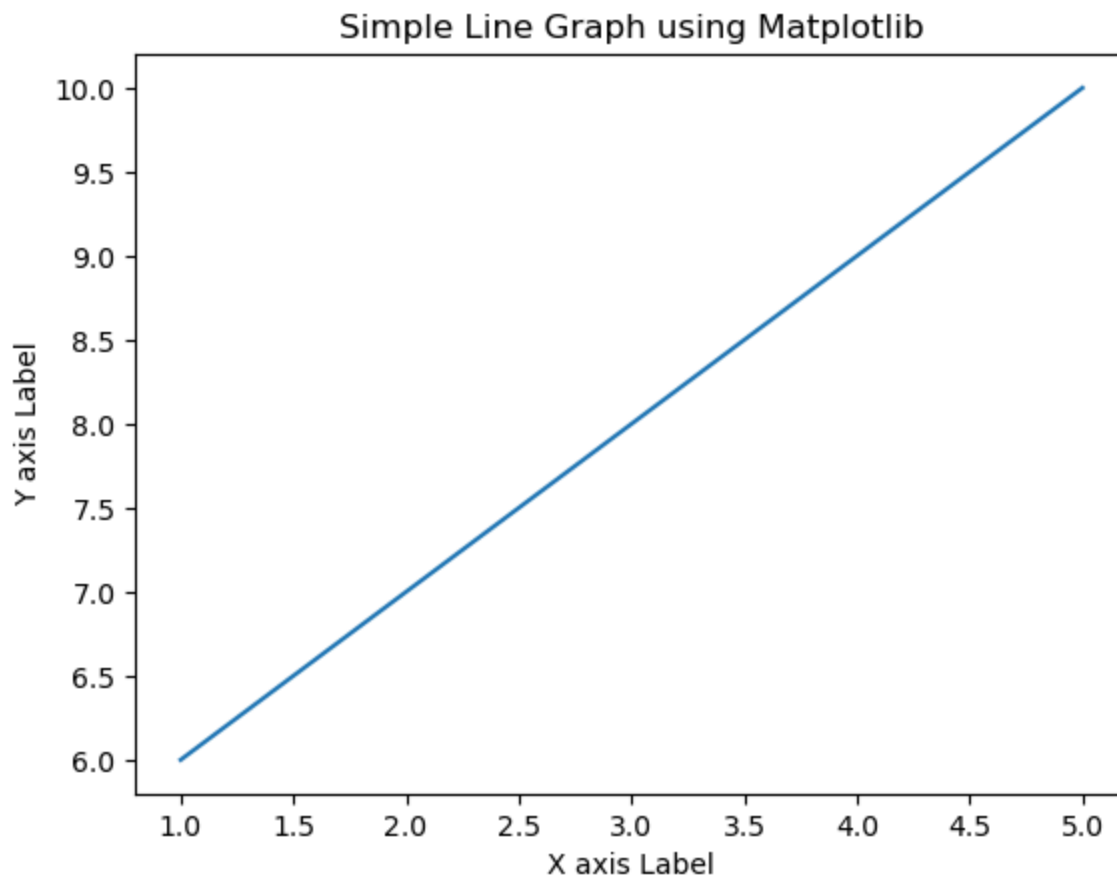
Matplotlib Visualization

In [49]: *# 1. Plot a simple line graph showing numbers 1 to 10 on both axes.*

```
x = [1,2,3,4,5]
y = [6,7,8,9,10]

plt.plot(x,y)
plt.title("Simple Line Graph using Matplotlib")
plt.xlabel("X axis Label")
plt.ylabel("Y axis Label")
```

Out[49]: Text(0, 0.5, 'Y axis Label')



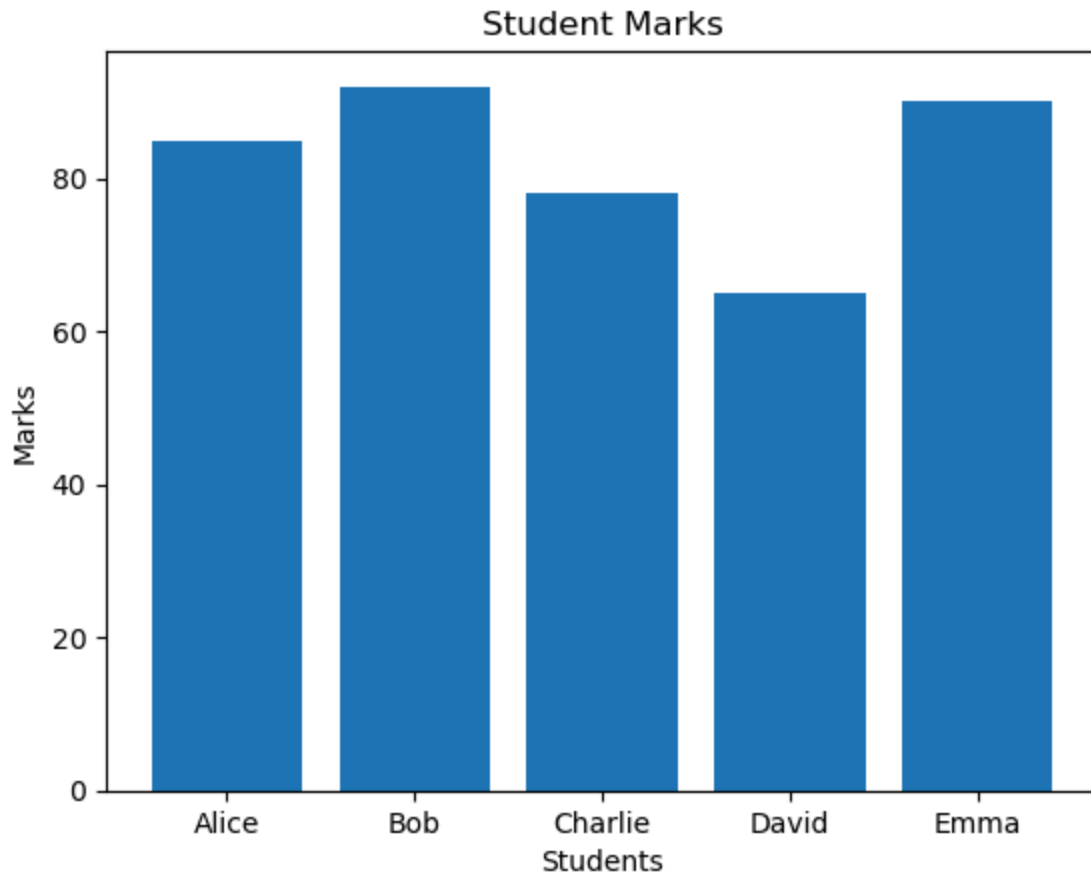
In [50]: *# 2. Create a bar chart showing marks of 5 students.*

```
students = ["Alice", "Bob", "Charlie", "David", "Emma"]
marks = [85, 92, 78, 65, 90]

plt.bar(students, marks)

plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")

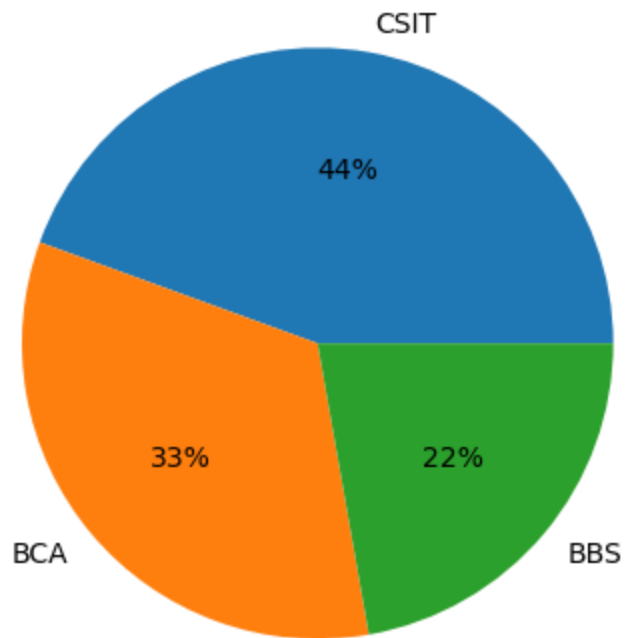
plt.show()
```



In [51]: *# 3. Draw a pie chart showing percentage of students in 3 departments.*

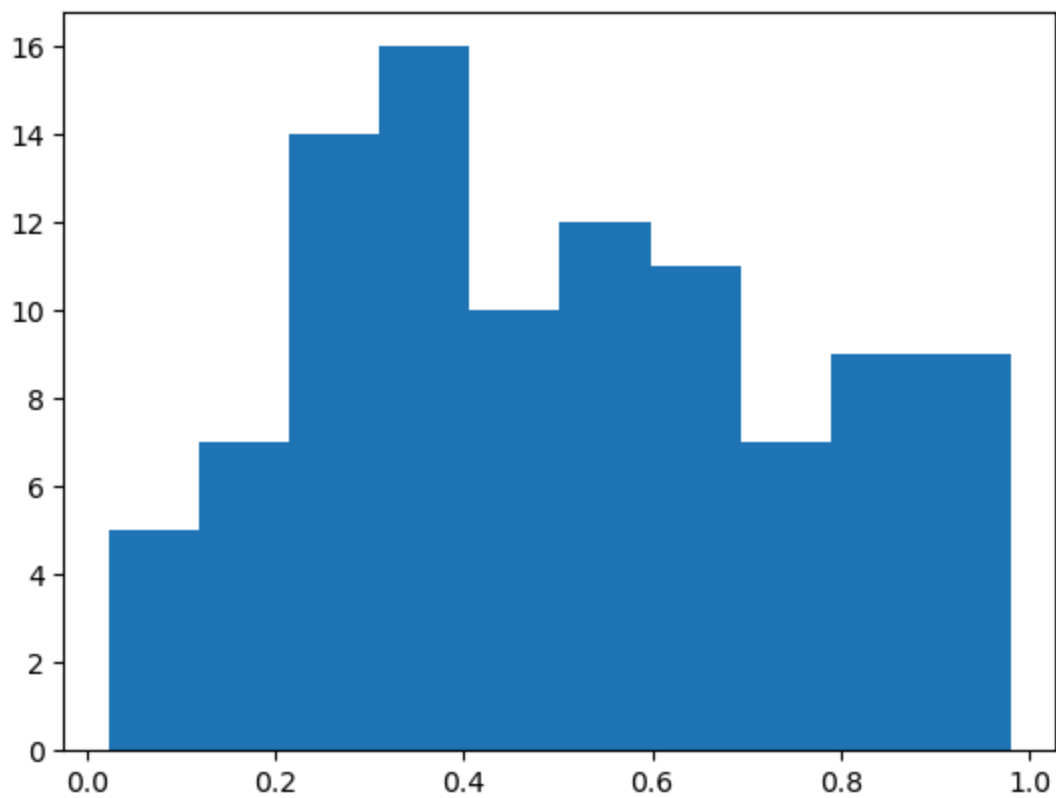
```
departments = ["CSIT", "BCA", "BBS"]  
num_students = [120, 90, 60]  
  
plt.pie(num_students, labels=departments, autopct="%d%%")  
plt.title("Students in Each Department")  
plt.show()
```


Students in Each Department



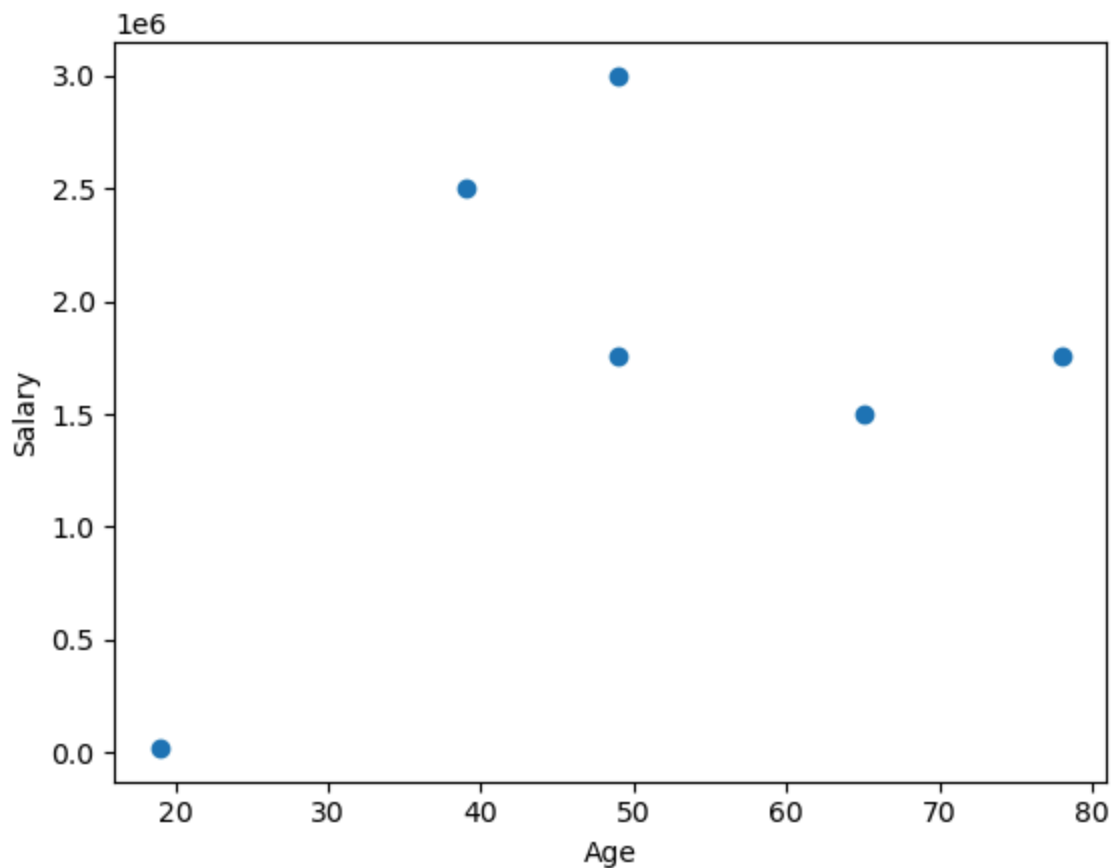
In [52]: *# 4. Plot a histogram for 100 random numbers to see the distribution.*

```
x = np.random.random(100)
plt.hist(x)
plt.show()
```



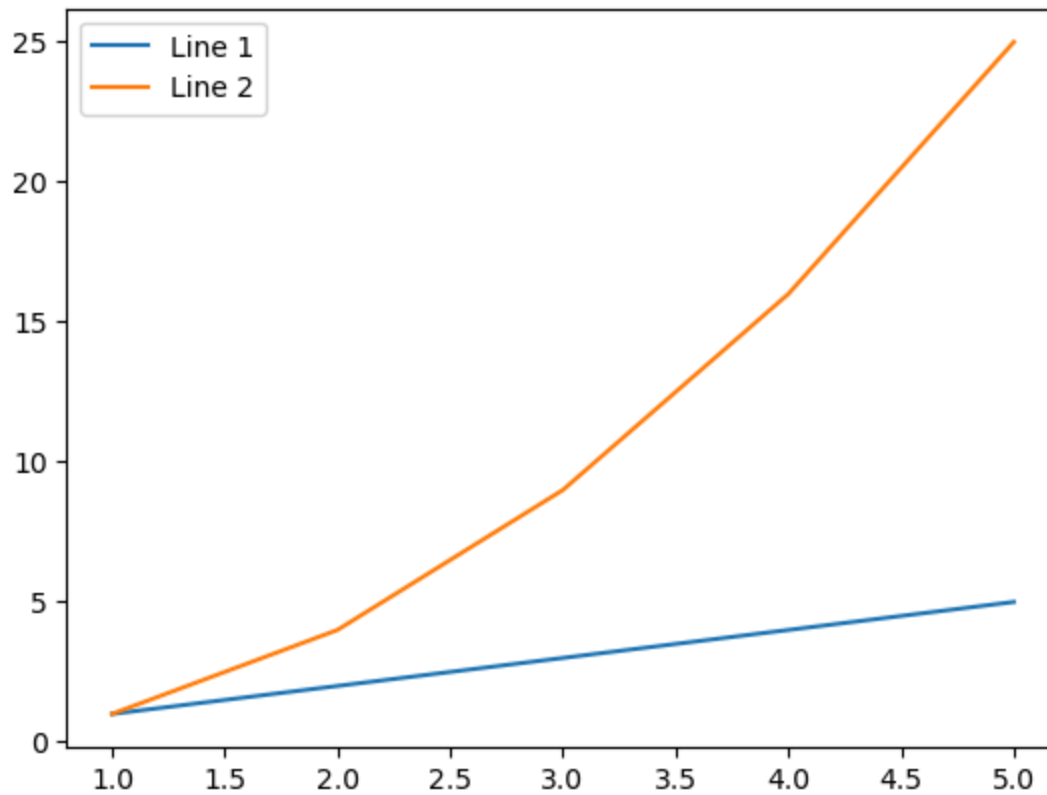
In [53]: *# 5. Create a scatter plot between Age and Salary from a sample dataset.*

```
plt.scatter(df['Age'], df['Salary'])  
plt.xlabel("Age")  
plt.ylabel("Salary")  
plt.show()
```



In [54]: *# 6. Plot two lines on the same graph and add a Legend.*

```
x = np.array([1, 2, 3, 4, 5])  
y1 = x  
y2 = x**2  
  
plt.plot(x, y1, label="Line 1")  
plt.plot(x, y2, label="Line 2")  
  
plt.legend()  
plt.show()
```

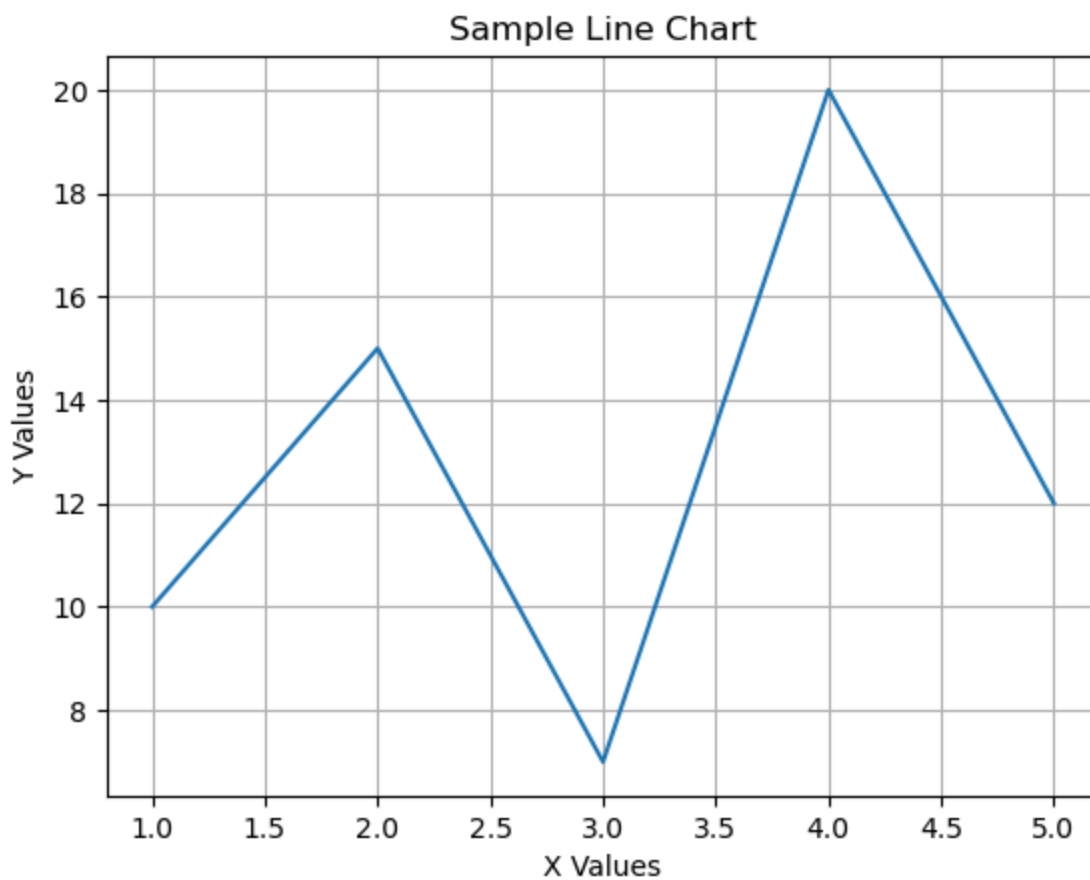


In [55]: # 7. Add title, x-label, y-label, and grid to any chart.

```
x = [1, 2, 3, 4, 5]
y = [10, 15, 7, 20, 12]

plt.plot(x, y)

plt.title("Sample Line Chart")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.show()
```



In [56]: *# 8. Use subplots to display a line chart and a bar chart side by side.*

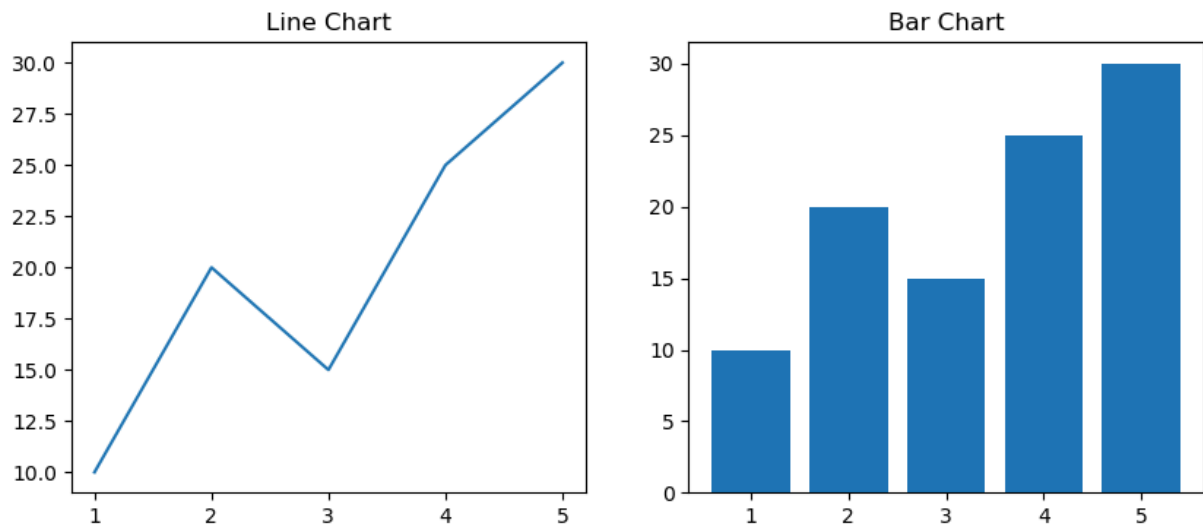
```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(x, y)
plt.title("Line Chart")

plt.subplot(1, 2, 2)
plt.bar(x, y)
plt.title("Bar Chart")

plt.show()
```



Reading & Cleaning CSV Files

In [75]: *# 1. Read a CSV file using pandas read_csv() and display its contents.*

```
df= pd.read_csv('Iris.csv')
df = df.drop(columns=['Id'])
df
```

Out[75]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

In [76]: *# 2. Print the column names, number of rows, and number of columns.*

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   SepalLengthCm   150 non-null   float64
1   SepalWidthCm    150 non-null   float64
2   PetalLengthCm   150 non-null   float64
3   PetalWidthCm    150 non-null   float64
4   Species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

In [77]: *# 3. Display the first 5 rows and check for missing values.*

```
print(df.head(5))
print(df.isnull().sum())
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
SepalLengthCm	0				
SepalWidthCm	0				
PetalLengthCm	0				
PetalWidthCm	0				
Species	0				

dtype: int64

In [78]: *# 4. Drop rows that contain any missing values using dropna().*

```
df_cleaned = df.dropna()
df_cleaned
```

Out[78]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
In [88]: # 5. Fill missing numeric values with the column mean using fillna()
numeric_cols = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

# verify there are no missing values left
print(df.isnull().sum())
```

```
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

```
In [ ]: # 6. Remove duplicate rows from the dataset.
print(df.duplicated().sum())
df = df.drop_duplicates()

# optional: reset the index after dropping rows
df = df.reset_index(drop=True)

print(df.shape)
```

```
0
(147, 5)
```

```
In [92]: # 7. Rename column headers to lowercase using str.lower().
```

```
df.columns = df.columns.str.lower()
print(df.columns)
```

```
Index(['sepalengthcm', 'sepalwidthcm', 'petallengthcm', 'petalwidthcm',
      'species'],
      dtype='object')
```

In [95]: *# 8. Convert a categorical column (e.g. Gender: Male/Female) into numbers using map*

```
df["species"] = df["species"].replace({
    "Iris-setosa": 0,
    "Iris-versicolor": 1,
    "Iris-virginica": 2
})
df
```

Out[95]:

	sepalengthcm	sepalwidthcm	petallengthcm	petalwidthcm	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
142	6.7	3.0	5.2	2.3	2
143	6.3	2.5	5.0	1.9	2
144	6.5	3.0	5.2	2.0	2
145	6.2	3.4	5.4	2.3	2
146	5.9	3.0	5.1	1.8	2

147 rows × 5 columns

In [97]: *# 9. Select only numeric columns and print their correlation using corr().*

```
print(df.select_dtypes(include=['number']).corr())
```


	sepalwidthcm	sepalwidthcm	petalwidthcm	petalwidthcm	\
sepalwidthcm	1.000000	-0.109321	0.871305	0.817058	
sepalwidthcm	-0.109321	1.000000	-0.421057	-0.356376	
petalwidthcm	0.871305	-0.421057	1.000000	0.961883	
petalwidthcm	0.817058	-0.356376	0.961883	1.000000	
species	0.782904	-0.418348	0.948339	0.955693	

	species
sepalwidthcm	0.782904
sepalwidthcm	-0.418348
petalwidthcm	0.948339
petalwidthcm	0.955693
species	1.000000

In [100... *# 10. Save the cleaned DataFrame to a new CSV file using to_csv().*

```
df.to_csv('cleaned_iris.csv', index=False)

print("DataFrame successfully saved as 'cleaned_iris.csv'")
```

DataFrame successfully saved as 'cleaned_iris.csv'

In []:

In []: